

2. Arquitectura por capas

Frontend

- **React 18 + TypeScript + Vite** (SPA).
- Librerías: React Router (navegación), Axios (cliente HTTP con interceptor de token JWT), componentes de interfaz propios con CSS modular, y Chart.js para el dashboard.
- Vistas principales: login, gestión de usuarios, carga de documentos, listado con estados, detalle/corrección de documento, búsqueda, chat conversacional (RAG), resumen y dashboard.
- Guard de rutas por rol (RBAC) en el frontend; el backend refuerza las autorizaciones.

Backend

- **FastAPI (Python 3.11)**, API REST con documentación automática en `/docs` (Swagger).
- Autenticación JWT (access token 30 min, renovable), contraseñas cifradas con BCrypt.
- Capas: `routes` (endpoints), `services` (lógica de negocio), `repositories` (acceso a datos).
- Validación de archivos por extensión, tipo MIME y magic bytes; límite de 25 MB.
- Registro de auditoría (log por usuario/acción/documento).
- Orquestación del pipeline de IA vía cola interna: al cargar un documento se encola su procesamiento y el estado avanza (pendiente → procesando → procesado/error).

Datos

- **PostgreSQL 16** con la extensión **pgvector**.
- Tablas relacionales: `usuarios`, `roles`, `categorias`, `documentos`, `metadatos`, `procesamiento`, `consultas`, `auditoria`.
- Columna `embedding vector(1536)` en `documentos` para la búsqueda semántica.
- Índice HNSW para búsqueda vectorial rápida y filtros por categoría.

Almacenamiento (archivos)

- Archivos originales en **MinIO** (compatible S3) dentro del ambiente Docker, o en una carpeta local configurable por variable de entorno en modo de desarrollo.
- El documento original se conserva intacto; el texto extraído y los vectores viven en PostgreSQL.

Componente de IA

- Módulo Python `ia/` con cuatro servicios internos:

1. **OCR:** PaddleOCR/Tesseract para imágenes y PDFs escaneados; extracción nativa con PyMuPDF/pdfplumber para PDFs digitales y `python-docx` para DOCX.
 2. **Chunking:** división del texto en fragmentos de ~512 tokens con solapamiento de 64.
 3. **Embeddings:** generación de vectores con `text-embedding-3-small` (1536 dims).
 4. **RAG/LLM:** recuperación de fragmentos por similitud coseno + generación de respuesta con `gpt-4o-mini`, incluyendo citas de las fuentes.
- Clasificación y resumen por prompting del LLM sobre el contenido extraído.